

Server-Side Multi-Parent Provenance Reconstruction for Copyright-Related Source Attribution in Heterogeneous Game MOD Packages

Anonymous Author(s)

Anonymous Institution

Abstract. Game MOD packages contain heterogeneous components such as Java bytecode, structured resources, and images, and may recombine parts of multiple prior MODs, including content from sources absent from the analysis gallery. Copyright-related source review therefore requires technical attribution at component level without assuming that a package has only one source. Pairwise clone detection or whole-package similarity alone cannot simultaneously explain component origins and package-level multi-parent lineage.

To support copyright source attribution, we propose a server-side Hierarchical Multi-Parent Provenance Reconstruction framework that assigns components to registered parents or an open-set UNKNOWN class while reconstructing the package-level parent set. The method extracts identity-neutral content evidence from CODE_BINARY, STRUCTURED, and IMAGE components without direct identifiers such as paths, project names, or namespaces, then performs global parent retrieval, bounded parent-set inference, and hierarchical component assignment. Dependency topology is limited to an optional refinement inside the candidate set.

Using current and historical releases of real public MOD projects, we construct a 120-project corpus and 540 frozen queries. The final TEST contains 360 queries and 2,520 components across K1, K2, K3, mixed known/unknown, and unknown-only settings. The final method achieves component accuracy 0.805952, UNKNOWN F1 0.753786, and parent-set F1 0.844233. Hierarchical content-only reconstruction improves exact parent-set match from 0.286111 to 0.413889 over independent attribution, whereas the graph-refinement parent-set F1 increment is only +0.000688 and is not statistically conclusive. A FastAPI-based materialized end-to-end pipeline achieves p50 latency of 26.88 ms, and 325 of 489 component errors (66.46%) occur during component assignment after retrieval.

The system does not automatically determine copyright ownership or infringement. It reconstructs technical provenance between uploaded MOD components and registered reference content to provide evidence for human review of reuse and copyright-related source attribution.

Keywords: Copyright-Related Source Attribution · Software Provenance · Game MOD · Multi-Parent Reconstruction · Open-Set Attribution · Software Reuse

1 Introduction

Game development and MOD creation frequently reuse code, configuration data, and media assets. Such reuse is not always recorded by package managers or explicit dependency metadata, and file-level copying can remove origin and version information while preserving technical similarity [1, 2]. For copyright-related source review of game MODs, one package may contain Java classes, structured resources, and images from different projects, while some sources may be absent from the server gallery. The technical task is therefore to attribute heterogeneous components, reconstruct multiple registered parents, and retain unexplained content as open-set UNKNOWN.

Clone detection and software composition analysis are strong at code/library reuse but typically return clone pairs or library matches [3–6]. Recent provenance studies address model merges, agent skills, and software-supply-chain telemetry [7–9]. We instead study joint component-level registered-parent/UNKNOWN attribution and package-level multi-parent reconstruction for heterogeneous game MOD packages.

We propose Server-Side Hierarchical Multi-Parent Provenance Reconstruction. Identity-neutral CODE_BINARY, STRUCTURED, and IMAGE evidence is extracted without paths, project names, or namespaces; candidate parents are retrieved globally; and a bounded package-level parent set constrains component assignment. Dependency topology is only an optional refinement. From 120 real public MOD projects and historical releases, we construct 540 frozen queries with project-level calibration, registered TEST, and held-out UNKNOWN splits. On the 360-query TEST, the final method reaches parent-set F1 0.844233 and is additionally evaluated with an external source baseline, a FastAPI server, multi-UNKNOWN stress, retrieval scaling, and failure localization.

Our contributions are:

1. an open-set multi-parent formulation for copyright-related attribution of heterogeneous MOD components and package-level lineage;
2. a hierarchical identity-neutral procedure separating retrieval, parent-set inference, component assignment, and graph refinement;
3. a frozen real-MOD benchmark with project-level splits, held-out UNKNOWN sources, leakage controls, and exact provenance; and
4. evaluation of accuracy, uncertainty, ablations, NiCadCross, server latency, multi-UNKNOWN robustness, Exact-versus-LSH scaling, and failure localization.

The output is technical provenance evidence, not an automatic legal determination of copyright ownership or infringement.

2 Background and Related Work

2.1 Code Similarity, Clone Detection, and Software Composition Analysis

Code similarity spans text, token, AST, graph, and learned representations; a systematic review reports persistent benchmark, evaluation, and language-support limitations [10]. NiCad uses source normalization for near-miss clones [3], SourcererCC uses token filtering/indexing at scale [4], and transformer-based clone detection has incorporated interpretability [11]. At package/library level, JC-Finder detects Java clone-based TPL reuse [6], BinCoFer targets partial C/C++ binary TPL reuse [12], and Android work highlights obfuscation/version challenges and dependency-fusion approaches [13, 14]. These methods identify reuse but do not jointly reconstruct multiple project parents and an open-set unknown class inside one heterogeneous package.

2.2 Game-Software Reuse and Copyright-Related Source Review

Game reuse can combine scripts, prefabs, assets, and scene-related resources, making the practical reuse unit broader than a single source file [1]. For copyright-related review, we preserve executable, structured, and decoded-image evidence as analyst-facing source evidence rather than a legal determination.

2.3 Provenance, Lineage, and Multi-Source Reconstruction

File copying creates implicit dependencies outside package-manager provenance [2]. Other multi-source lineage systems verify model parents and merges [7], audit mixed-modality agent-skill reuse [8], or reconstruct runtime supply-chain provenance from heterogeneous telemetry [9]. Our novelty is not the terms “heterogeneous provenance” or “multi-parent” themselves, but their concrete combination with component-level registered-parent/UNKNOWN attribution, package-level reconstruction, a frozen real-MOD benchmark, and server-side evaluation.

3 System and Problem Model

3.1 Server-Side Analysis Scenario

The analysis server maintains previously registered reference MODs. Registration decomposes each package, extracts identity-neutral evidence, and builds a parent-indexed gallery. Online analysis applies the same extractors to an uploaded archive and returns the reconstructed parent set, component labels, and UNKNOWN decisions for human comparison; it makes no legal determination.

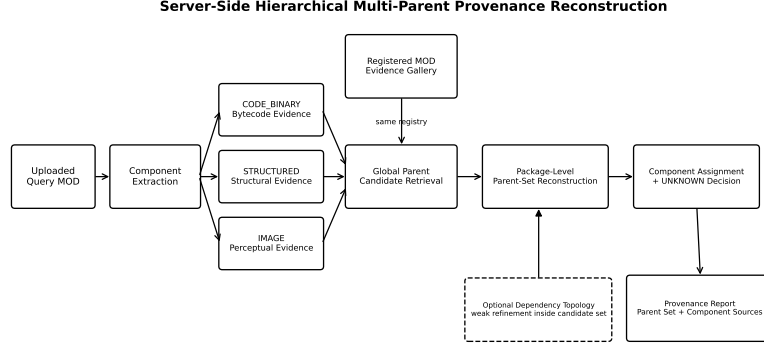


Fig. 1. Online provenance reconstruction flow. Identity-neutral content evidence from CODE, STRUCTURED, and IMAGE components is the primary signal; dependency topology is used only as an optional refinement inside the candidate set.

3.2 Multi-Parent Open-Set Formulation

Let the registered reference gallery be

$$\mathcal{G} = \{G_1, \dots, G_N\}.$$

A query MOD is represented as

$$G_q = (V_q, E_q),$$

where V_q is the component set and E_q is an optional dependency relation. Each component $v \in V_q$ has modality

$$m(v) \in \{\text{CODE}, \text{STRUCTURED}, \text{IMAGE}\},$$

and provenance label

$$y_v \in \{1, \dots, N, \text{UNKNOWN}\}.$$

The package-level registered-parent set is

$$P_q = \{p \mid \exists v \in V_q, y_v = p, p \neq \text{UNKNOWN}\}.$$

The system returns component labels $\hat{y}_v$, a predicted parent set $\hat{P}_q$, and a registered-parent count $\hat{K} = |\hat{P}_q|$.

In this work, UNKNOWN does not mean “original work” or “novel content.” It only means that the current server-side registered gallery cannot explain the component. Even when multiple distinct unknown sources are present, the primary model exposes a single collapsed UNKNOWN label.

3.3 Analysis Assumptions and the Hard Evaluation Track

We do not assume a specific adversarial transformation strategy. Instead, we define a hard evaluation track so that the provenance model cannot rely on easy identity cues such as file paths or naming conventions. The final attribution method does not consume original relative paths, basenames, MOD namespaces, source-project identities, source SHA-256 values, or ground-truth labels. For Java code, the representation also excludes method/class names, descriptors, string constants, and constant-pool operands.

This design prevents trivial solutions that succeed only because names are shared across releases, and forces provenance evidence to come from content structure and behavior-related patterns. It does not imply robustness to every form of obfuscation or semantics-preserving transformation.

3.4 Structured Objective

Conceptually, the decision can be written as

$$\hat{\mathbf{y}} = \arg \min_{\mathbf{y}} \left[\sum_{v \in V_q} \phi_v(y_v) + \gamma \Omega(\mathbf{y}) + \beta \sum_{(u,v) \in E_q} \psi_{uv}(y_u, y_v) \right],$$

where ϕ_v is a heterogeneous content-based component-parent assignment cost, Ω penalizes unnecessary parent proliferation at package level, and ψ_{uv} is an optional dependency-consistency term. In implementation, we avoid direct combinatorial search over the full gallery by separating candidate retrieval from bounded parent-set search.

4 Proposed Method

4.1 Overall Processing Flow

The proposed method proceeds through seven stages: (1) component extraction, (2) modality-specific identity-neutral evidence extraction, (3) component-to-parent distance computation, (4) package-level candidate retrieval, (5) bounded parent-set reconstruction, (6) component assignment and UNKNOWN rejection, and (7) optional dependency refinement. The central design separates component evidence from package-level decisions. An independent classifier may allow one or two noisy components to introduce additional false parents, whereas the hierarchy first selects a compact parent set using evidence across the entire query.

4.2 CODE_BINARY Evidence

Java class files are parsed into method-level opcode sequences. The identity-neutral CODE representation uses three 128-bit SimHash signatures.

First, an opcode 3-gram representation preserves local opcode patterns without operands or constant values. Second, a structural representation summarizes

method length, maximum stack, maximum locals, access flags, opcode-category counts, and class-level method/field/instruction counts using log buckets. Third, a context representation uses 3-grams of opcode categories such as arithmetic, branch, load/store, and invoke. This preserves control/operation structure to a limited extent while excluding direct identity information such as class names, method names, descriptors, and string literals.

4.3 STRUCTURED Evidence

For JSON-like structured resources, the representation preserves structure and value shape rather than literal identifiers. During key/value traversal, object/array depth, key categories, numeric/string/boolean/null patterns, and sequence structure are converted into tokens and summarized as a 128-bit SimHash. If a resource identifier has the form `namespace:path`, an external namespace such as `minecraft` may be retained as a category, while MOD-specific namespace literals are not exposed. Long hexadecimal strings, URLs, and qualified identifiers are likewise mapped to shape categories rather than retained literally.

Trivial structured components that recur across many projects and carry little provenance information, such as empty JSON objects, are removed during benchmark preprocessing.

4.4 IMAGE Evidence

IMAGE evidence is computed from decoded pixels rather than file names or paths. We extract 64-bit aHash, dHash, and pHash signatures from grayscale images and compute a normalized 16-bin luminance histogram. These perceptual features can preserve similarity when byte-level SHA changes under lossless re-encoding. IMAGE payloads in the final benchmark are losslessly re-encoded as PNG, and decoded-pixel equality is verified.

4.5 Component-to-Parent Cost

Each query component is compared with gallery components of the same modality. Binary signatures use normalized Hamming distance, whereas image histograms use normalized L_1 distance. The component distance $d(v, p)$ to parent p is derived from the nearest same-modality evidence owned by that parent.

After normalizing representation-specific distances, we additionally use regret relative to the strongest visible parent. The frozen component cost is conceptually

$$c(v, p) = \alpha \tilde{d}(v, p) + (1 - \alpha) \tilde{r}(v, p),$$

with $\alpha = 0.75$ and regret weight 0.25. Modality-specific open-set thresholds are selected only on the calibration split.

4.6 Global Candidate Retrieval

Directly enumerating parent combinations over the entire registered gallery would grow rapidly. We therefore first compute a query-level retrieval score to reduce the search space. For query q and parent p , the retrieval score aggregates the $L = 3$ smallest component costs:

$$R_q(p) = \frac{1}{L} \sum_{i=1}^L c_{(i)}(p), \quad L = 3,$$

where $c_{(i)}(p)$ is the i th smallest query-component cost for parent p . This favors parents supported by several strong provenance signals instead of allowing a single accidental match to dominate ranking. The $M = 10$ parents with the smallest $R_q(p)$ are retained as candidate pool C_q . On TEST, mean registered-parent recall is 0.974444 and all true registered parents are present for 0.943333 of applicable queries.

4.7 Hierarchical Parent-Set Reconstruction

For candidate pool C_q , we enumerate parent subsets $S \subseteq C_q$ with $|S| \leq K_{\max} = 3$. Given S , each component chooses the lowest frozen assignment cost among $S \cup \{\text{UNKNOWN}\}$. The package objective combines component fit with a parent-count penalty:

$$J_q(S) = \sum_{v \in V_q} \min_{y \in S \cup \{\text{UNKNOWN}\}} c(v, y) + \lambda |S|,$$

$$\hat{S}_q = \arg \min_{S \subseteq C_q, |S| \leq K_{\max}} J_q(S), \quad \lambda = 0.5.$$

This hierarchy suppresses the tendency of component-level local optima to introduce too many parents in one query. The final graph variant adds a low-weight boundary-consistency term inside the selected subset, while the content-only variant uses the same candidate pool and package objective without the graph term.

Table 1. Implementation procedure for Hierarchical Multi-Parent Provenance Reconstruction.

Step	Procedure
1	Extract CODE_BINARY, STRUCTURED, and IMAGE components from the query package.
2	Compute identity-neutral evidence for each component.
3	Compare with same-modality gallery evidence to compute component-parent costs.
4	Aggregate query-level evidence and retrieve the top $M = 10$ registered parents.
5	Evaluate parent subsets with $ S \leq 3$ inside the candidate pool.
6	Assign each component to a registered parent or UNKNOWN for each subset.
7	Select the subset minimizing the package objective with parent-count penalty and optional graph term.
8	Return the final parent set, $\hat{K}$, component labels, and uncertainty-related evidence in the provenance report.

4.8 UNKNOWN Handling

Open-set thresholds are selected per modality on calibration data and frozen before TEST. The frozen thresholds are 0.1302083284 for CODE, 0.03125 for STRUCTURED, and 0 for IMAGE. The IMAGE threshold of 0 means that this specific frozen representation/calibration selected a highly conservative exact-nearest condition; it is not a universal threshold for other datasets.

The primary TEST contains at most one held-out unknown source per query. A later robustness analysis places two or three distinct held-out donors in the same query to test the stability of collapsed UNKNOWN rejection.

4.9 Optional Dependency Refinement

The dependency graph is dominated by Java class references, with sparse non-class edges. We therefore do not describe it as a fully heterogeneous provenance graph. Graph evidence only weakly refines selected-parent assignment within the candidate pool, with frozen weight $\beta = 0.1$. The CONNECTED_STRESS track adds the minimum deterministic bridges needed to connect the five code nodes without using parent labels or K , serving as a controlled stress test for cross-parent structural bridges. Consistent with the measured results, the graph is positioned as optional refinement rather than as the central contribution.

5 Benchmark Construction and Experimental Design

5.1 Real MOD Corpus and Historical Releases

The final corpus contains 120 real public MOD projects: 90 target projects and 30 background projects. For each project, we collect the current release and available historical releases and construct a component registry. Before filtering, the current registry contains 60,257 components and the historical registry contains 178,162 components.

Trivial structured components with little provenance signal, such as empty JSON objects, are removed in advance. After filtering, the current registry contains 56,035 components and the historical registry contains 165,496 components. Cross-project exact-SHA duplicates in the current corpus are limited to five groups covering 48 components. This audit is used to check whether repeated trivial resources could dominate the benchmark.

5.2 Identity-Leakage Audit

We analyze identity overlap between 124,470 historical target components and their current releases. Historical components whose exact SHA-256 does not appear in their own current release are defined as hard-masked candidates; 31,279 components satisfy this condition. A stricter subset of 6,429 components also has no identical full relative path in the current release.

Across TEST_KNOWN historical components, own exact-SHA survival is 0.677471 while same-path survival is 0.919728. This indicates that retaining paths or names would provide a substantially easier identity cue than content transformation. The final method therefore does not use paths, basenames, namespaces, or source-project identities.

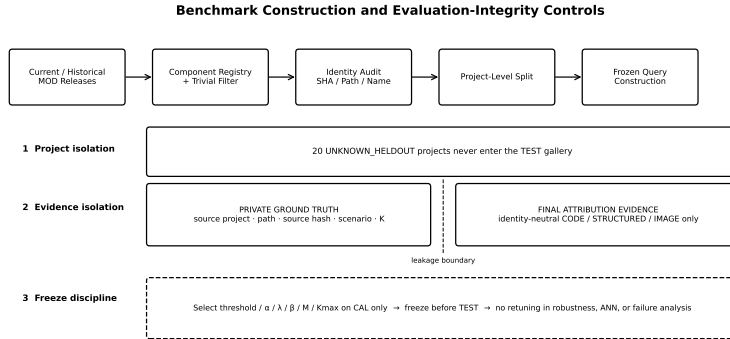


Fig. 2. Benchmark construction and evaluation-integrity controls. Project-level open-set isolation, separation of private ground truth from final attribution evidence, CAL-only tuning, and freezing before TEST are enforced together.

5.3 Project Split and Open-Set Isolation

Project splits are fixed before component-level sampling. This prevents highly similar releases from the same project from appearing in both calibration and TEST.

Table 2. Frozen project-level split.

Split	Projects		Role
CAL_KNOWN	25	Registered targets for calibration	
TEST_KNOWN	45	Registered targets for final TEST	
UNKNOWN_HELDOUT	20	Open-set sources absent from the gallery	
CAL_BACKGROUND	15	Calibration-gallery background	
TEST_BACKGROUND	15	TEST-gallery background	

The calibration gallery contains 40 projects and the TEST gallery contains 60 projects. None of the 20 UNKNOWN_HELDOUT projects enters the TEST gallery. After hard masking and fixed-modality-composition constraints, the actual donor pools contain 19 CAL_KNOWN projects, 33 TEST_KNOWN projects, and 15 UNKNOWN_HELDOUT projects.

5.4 Automatic Query Generation and Ground Truth

The final benchmark does not rely on manual labels indicating which MOD each file came from. Instead, the historical component registry and a private

source manifest automatically select donor projects and source components, and the exact component origin is stored in the private query manifest. Only public/materialized payloads that do not expose source identity are provided to the evaluation pipeline.

A total of 540 queries are generated: 180 for calibration and 360 for TEST. Each query contains exactly seven components:

$$5 \text{ CODE} + 1 \text{ STRUCTURED} + 1 \text{ IMAGE}.$$

The complete materialized benchmark therefore contains 3,780 payloads. All 540 IMAGE payloads are losslessly re-encoded as PNG and decoded-pixel equality is verified. Source-hash verification passes for all 3,780 payloads.

TEST contains 60 queries for each of the six scenarios in Table 3.

Table 3. Scenario composition of the frozen TEST.

Scenario	Queries	Registered parents	Unknown sources
TEST_KNOWN_K1	60	1	0
TEST_KNOWN_K2	60	2	0
TEST_KNOWN_K3	60	3	0
TEST_MIXED_1K1U	60	1	1
TEST_MIXED_2K1U	60	2	1
TEST_UNKNOWN_K1	60	0	1

Using actual total-source count, $K=1$, $K=2$, and $K=3$ each account for 120 queries. The public manifest does not expose K , scenario, source identity, path, or source hash.

5.5 Graph Track

For the five selected CODE components, the NATURAL graph preserves historical CLASS_REF edges. A separate CONNECTED_STRESS track connects the five code nodes by adding the minimum deterministic bridges without using parent labels or K . This track is not presented as an empirical natural graph; it is a controlled structural stress test of graph-based grouping under cross-parent bridges.

5.6 Calibration and Freeze Discipline

All thresholds and reconstruction hyperparameters are selected only on calibration data and are not changed after TEST predictions are observed.

The frozen configuration is: CODE threshold 0.1302083283662796, STRUCTURED threshold 0.03125, IMAGE threshold 0, content weight $\alpha = 0.75$, regret weight 0.25, parent penalty $\lambda = 0.5$, candidate size $M = 10$, graph weight $\beta = 0.1$, boundary Top- $R = 3$, and $K_{\max} = 3$. Later robustness, scalability, and failure analyses do not retune these values.

5.7 Statistical Analysis and Reproducibility Protocol

Primary uncertainty is estimated with 10,000 scenario-stratified bootstrap replicates while preserving all seven components of a query as one cluster. Graph-versus-content comparison uses paired bootstrap deltas on the same queries. To examine repeated use of source projects, we additionally perform source-anchor cluster bootstrap and leave-one-source-project-out sensitivity analysis. Parameters are selected on CAL only; TEST is not used for tuning.

When all 2,520 materialized TEST components are re-extracted from raw payloads, every payload SHA-256 matches and all 40,320 compared evidence fields have zero mismatches. Query evidence tables exclude source paths, project IDs, ground-truth labels, scenarios, K, and source hashes, and a programmatic check confirms that UNKNOWN_HELDOUT projects are absent from the gallery.

5.8 Experimental Environment

Primary accuracy evaluation and server benchmarking use the same conceptual pipeline but measure different properties, so their performance values are reported separately. The server host and benchmark protocol are summarized in Table 4.

Table 4. Server-side benchmark environment and protocol.

Item	Configuration
Host OS	Windows 10, build 26200
Python	3.10.6
CPU	20 physical / 28 logical cores
Server	FastAPI / Uvicorn, single process
TEST gallery	60 projects, 26,329 components
Concurrency	1, 4, 8, 16, 32
Requests	200 requests per concurrency level
Warm-up	20 requests
Random seed	20260813
Input	Local materialized query ZIP; network upload time excluded

The external source baseline uses Open-NiCad/NiCadCross in a WSL Linux environment. StoneDetector is treated as a compatibility audit because the official implementation encounters modern Java bytecode/toolchain issues; we do not over-interpret it as a performance baseline.

5.9 Metrics and Research Questions

At component level, we report accuracy and UNKNOWN precision/recall/F1; at package level, parent-set precision/recall/F1, exact match, K accuracy, and K MAE; and at retrieval level, registered-parent recall. Six questions guide evaluation: **RQ1**, how accurate is copyright-related source reconstruction? **RQ2**, what do hierarchy and graph refinement contribute? **RQ3**, how does the method compare with a compatible external clone baseline? **RQ4**, does the server reproduce frozen decisions with practical performance? **RQ5**, is it robust to multiple unregistered sources and approximate retrieval? **RQ6**, where do residual errors arise?

6 Experimental Results

6.1 RQ1: Primary Performance on the Frozen TEST

Table 5. Final performance on the frozen primary TEST.

Metric	Score
Component Accuracy	0.805952
UNKNOWN Precision	0.652427
UNKNOWN Recall	0.892430
UNKNOWN F1	0.753786
Parent-Set F1	0.844233
Exact Parent-Set Match	0.419444
K Accuracy	0.486111
K MAE	0.527778
Candidate mean registered-parent recall	0.974444
All true registered parents present	0.943333

The final method attains component accuracy 0.805952, UNKNOWN F1 0.753786, and parent-set F1 0.844233. Exact match (0.419444) and K accuracy (0.486111) show that exact source-count recovery remains difficult.

6.2 Performance by Scenario

Table 6. Final results by frozen TEST scenario. Each scenario contains 60 queries.

Scenario	Comp. Acc.	Parent F1	Exact	K Acc.	UNK F1
Known K1	0.757143	0.750000	0.283333	0.283333	–
Known K2	0.809524	0.843333	0.316667	0.383333	–
Known K3	0.745238	0.787937	0.100000	0.333333	–
Mixed 1K1U	0.828571	0.930556	0.700000	0.716667	0.847059
Mixed 2K1U	0.797619	0.917460	0.600000	0.683333	0.739550
UNKNOWN-only	0.897619	0.836111	0.516667	0.516667	0.946048

Known K3 has the lowest exact match (0.100000), consistent with the difficulty of recovering all three registered parents simultaneously. Higher mixed-scenario parent-set F1 is interpreted descriptively because source shares differ and unseen sources collapse to one UNKNOWN label.

6.3 Performance by Modality

Table 7. Final component results by modality on the frozen TEST.

Modality	Components	Comp. Acc.	UNK Recall	UNK F1
CODE_BINARY	1,800	0.810000	0.893773	0.766091
IMAGE	360	0.972222	0.911765	0.948980
STRUCTURED	360	0.619444	0.866667	0.581470

IMAGE is most stable, CODE_BINARY is intermediate, and STRUCTURED is hardest. Similar structural shapes after identifier removal motivate stronger structured-resource evidence and modality-specific calibration.

6.4 Uncertainty: Query Bootstrap and Source-Project Sensitivity

A 10,000-replicate scenario-stratified query bootstrap gives 95% intervals [0.787698, 0.823810] for component accuracy, [0.829101, 0.859114] for parent-set F1, [0.372222, 0.466667] for exact match, [0.436111, 0.533333] for K accuracy, and [0.732551, 0.774445] for UNKNOWN F1. Source-anchor bootstrap gives [0.820531, 0.867090] for parent-set F1; leave-one-source-project-out changes remain small (maximum 0.005836 for parent-set F1 and 0.011392 for component accuracy).

6.5 RQ2: Effect of Hierarchical Reconstruction

Table 8. Independent attribution, hierarchical content reconstruction, and final graph refinement.

Method	Comp. Acc.	UNK F1	Parent F1	Exact	K Acc.
Independent	0.775794	0.741806	0.797412	0.286111	0.325000
Hierarchical content	0.807143	0.752386	0.843545	0.413889	0.480556
Final + graph	0.805952	0.753786	0.844233	0.419444	0.486111
Oracle-K content	0.825794	0.790948	0.872976	0.558333	0.627778

Hierarchical content-only reconstruction improves component accuracy by +0.031349, parent-set F1 by +0.046133, exact match by +0.127778, and K accuracy by +0.155556 over independent attribution. The graph changes parent-set F1 by only +0.000688 and its paired-bootstrap 95% interval [-0.001693, 0.003466] includes zero. Oracle-K raises exact match to 0.558333, identifying K inference as a residual bottleneck.

6.6 RQ3: External Source-Level Baseline

NiCadCross is evaluated on 1,169 source-resolvable TEST CODE components; 3,275/10,448 gallery CODE components are source-resolvable. With functions/Java/default blind-renaming and threshold 0.30, 10 query and 19 gallery parses fail, yielding 2,199 clone pairs and 505 matched classes. Paired

Table 9. NiCadCross and the proposed method on the same 1,169 source-resolvable components.

Method	Comp. Acc.	UNK F1	Parent F1	Exact	K Acc.
NiCadCross	0.710009	0.678673	0.815468	0.725490	0.588235
Proposed	0.840890	0.794016	0.867429	0.787582	0.771242

query-bootstrap intervals favor the proposed method for component accuracy ([0.096187, 0.166524]), UNKNOWN F1 ([0.079256, 0.154288]), and K accuracy ([0.114379, 0.251634]); exact-match delta includes zero. Because NiCad receives reconstructed Java source, this is a same-subset comparison, not a universal-superiority claim. StoneDetector is retained only as a compatibility audit due to modern bytecode/toolchain failures.

6.7 RQ4: Real Server-Side Evaluation

Offline-Server Correctness. The single-process FastAPI/Uvicorn server reproduces candidate top-10 results, selected subsets, K, and all 2,520 assignments for all 360/360 frozen TEST queries.

Materialized End-to-End Latency. The benchmark includes archive/manifest parsing, payload reading, raw evidence extraction, 60-project gallery search, reconstruction, graph refinement, and response construction; network upload is excluded. At concurrency 1, total p50 is 26.880 ms; extraction (14.440 ms) and

Table 10. p50 latency breakdown for a single materialized-package request.

Stage	p50 latency
Archive open + manifest	0.334 ms
Payload read	0.232 ms
Identity-neutral extraction	14.440 ms
60-project gallery search	10.685 ms
Reconstruction	1.086 ms
Total server processing	26.880 ms

search (10.685 ms) dominate reconstruction (1.086 ms).

Concurrency and Throughput. Across 200 requests at each concurrency 1, 4, 8, 16, and 32, all 1,000 requests succeed. Throughput is 24.72, 37.60, 38.00, 38.71, and 37.16 req/s, peaking at concurrency 16 before single-process contention dominates.

Real-Project Gallery Scaling. Increasing the real gallery from 20 to 100 projects grows components 16,282 to 48,376 ($2.97\times$), search p50 4.353 to 21.457 ms ($4.93\times$), and reconstruction p50 only 0.680 to 1.423 ms. Gallery search is the main scale-up target.

6.8 RQ5: Multi-UNKNOWN Robustness and Approximate Retrieval

Controlled Multi-UNKNOWN Stress. A post-freeze 180-query stress test with two or three distinct held-out donors achieves component accuracy 0.841270, UNKNOWN F1 0.887991, collapsed parent-set F1 0.883968, exact match 0.600000, and collapsed K accuracy 0.633333. On 120 queries containing registered parents, registered-parent precision/recall/F1 are 0.815278/0.912500/0.840556 with exact match 0.625000. Rejection remains stable, but unknown identities and multiplicity are not recovered.

Exact-versus-LSH Retrieval. BALANCED and HIGH_RECALL reproduce all Exact content-only predictions; FAST changes only 1/360 query and 1/2,520 components (component accuracy 0.806746 vs. 0.807143; parent-set F1 0.842804 vs. 0.843545).

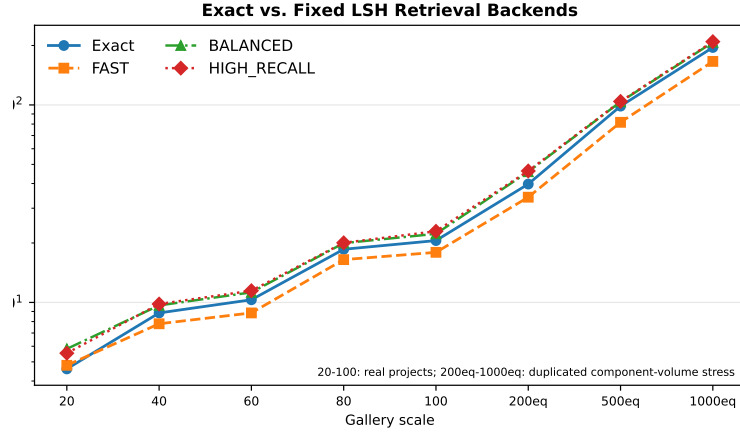


Fig. 3. Retrieval p50 for Exact and three fixed LSH configurations. The 20–100 points use real registered-project galleries; 200eq–1000eq are computational component-volume stresses obtained by duplicating arrays from the same 100 real parents.

On the 60-project runtime sample, FAST reduces p50 from 10.315 to 8.848 ms ($1.166\times$); fidelity-preserving LSH settings show no p50 advantage over Exact at 20–100 real projects.

6.9 RQ6: Post-Freeze Failure Localization

Of 2,520 components, 489 are incorrect: 358 known-to-UNKNOWN, 81 UNKNOWN-to-known, and 50 wrong-known-parent transitions.

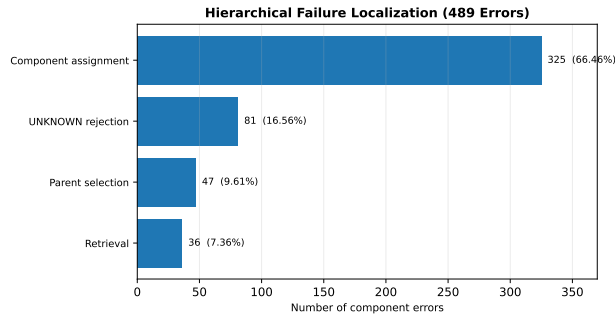


Fig. 4. Hierarchical post-freeze failure localization for all 489 component errors in the frozen TEST.

Component Assignment Miss dominates with 325 errors, while Retrieval Miss accounts for 36. At query level, 209 parent-set failures split into 192 reconstruction-limited and 17 retrieval-limited cases. All 102 TEST_KNOWN_K1 errors are

assignment failures; K3 additionally shows 21 retrieval and 19 parent-selection misses. Assignment/UNKNOWN calibration and K inference therefore have higher priority than simply increasing candidate recall.

7 Discussion

The ablation identifies hierarchy, not graph refinement, as the main gain: content-only reconstruction improves parent-set F1, exact match, and K accuracy substantially, whereas the graph increment is statistically inconclusive. Of 489 errors, 325 occur after the true parent survives retrieval and package-level selection, and Oracle-K raises exact match from 0.413889 to 0.558333; assignment calibration and source-count inference are thus the main residual bottlenecks. The output remains analyst-facing provenance that narrows candidates and localizes attribution. No score establishes authorship, ownership, copying, permission, license compliance, or infringement, and UNKNOWN only denotes insufficient gallery support.

8 Limitations and Threats to Validity

Controlled recombination yields exact donor labels but does not reproduce all real refactoring, partial edits, image transformations, or resource merges. Because the benchmark is not adjudicated copyright cases, its metrics measure provenance reconstruction, not legal-decision accuracy. Leakage controls remove identity cues and re-extraction shows no mismatch. External validity is limited by the 120-project Java-centered corpus; 200eq–1000eq are computational-volume stresses, not independent real projects. The graph delta is not significant, NiCadCross is source-subset only, and StoneDetector is a compatibility audit. The model exposes one collapsed UNKNOWN, the graph is mainly a Java class-reference backbone, and host-specific latency excludes network upload. Post-freeze analyses do not retune the primary TEST.

9 Conclusion

We presented server-side Hierarchical Multi-Parent Provenance Reconstruction for copyright-related source attribution in heterogeneous game MOD packages. From 120 real projects and historical releases, we build 540 frozen queries and 3,780 materialized components; on 360 TEST queries the method achieves component accuracy 0.805952, UNKNOWN F1 0.753786, and parent-set F1 0.844233. Hierarchical reconstruction provides the main gain, while graph improvement is inconclusive. The FastAPI implementation reproduces all frozen outputs at 26.88 ms materialized p50. Future work should prioritize assignment calibration, structured-resource evidence, and source-count inference. The system supports human copyright-related source review, not legal determination.

References

1. Trasobares Ibor, J.I., Domingo-Montes, A., Casamayor-Moragriega, R., Blasco-Latorre, D., Cetina, C.: A multiple case study on reuse in game software engineering. *Information and Software Technology* **185**, 107781 (2025). <https://doi.org/10.1016/j.infsof.2025.107781>
2. He, R., Mockus, A., Yang, W., Zhou, M.: File-level copying is an implicit dependency in open source. arXiv preprint arXiv:2607.02059 (2026)
3. Roy, C.K., Cordy, J.R.: NICAD: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In: 16th IEEE International Conference on Program Comprehension. pp. 172–181 (2008). <https://doi.org/10.1109/ICPC.2008.41>
4. Sajjani, H., Saini, V., Svajlenko, J., Roy, C.K., Lopes, C.V.: Sourcerercc: Scaling code clone detection to big code. In: Proceedings of the 38th International Conference on Software Engineering (2016)
5. Heinze, T.S., Schäfer, A., Amme, W.: Stonedetector: Conventional and versatile code clone detection for java. *Journal of Systems and Software* **236**, 112799 (2026). <https://doi.org/10.1016/j.jss.2026.112799>
6. Zhao, L., Li, C., Wu, Y., Zhang, L., Wu, J., Liu, C., Chen, S., Hu, Y., Xu, Z., Liu, Y., Ge, J., Sun, J., Liu, Y.: JC-Finder: Detecting java clone-based third-party library by class-level tree analysis. arXiv preprint arXiv:2508.02397 (2025)
7. Adil, M.A.B., Aamir, S.: modeldna: Calibrated lineage verification and merge decomposition from sampled weight fingerprints. arXiv preprint arXiv:2607.10617 (2026)
8. Chen, J., Wang, M., Jiang, L., Ma, J., Deng, X., Du, X., Lin, R., Feng, Y., Du, L., Wang, J.: Skilltrace: Multi-trace provenance auditing for llm-agent skill reuse. arXiv preprint arXiv:2608.05204 (2026)
9. Tan, Z., Tang, Y., Singer, J., Anagnostopoulos, C., Xiao, K.: Fusechain: Runtime evidence reconstruction for software supply-chain attacks. arXiv preprint arXiv:2606.15811 (2026)
10. Zakeri-Nasrabadi, M., Parsa, S., Ramezani, M., Roy, C., Ekhtiarzadeh, M.: A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. *Journal of Systems and Software* **204**, 111796 (2023). <https://doi.org/10.1016/j.jss.2023.111796>
11. Nashaat, M., Amin, R., Eid, A.H., Abdel-Kader, R.F.: An enhanced transformer-based framework for interpretable code clone detection. *Journal of Systems and Software* **222**, 112347 (2025). <https://doi.org/10.1016/j.jss.2025.112347>
12. Zou, Y., Zhang, Y., Zhao, G., Wu, Y., Shen, S., Fu, C.: Bincofer: Three-stage purification for effective c/c++ binary third-party library detection. *Journal of Systems and Software* **229**, 112480 (2025). <https://doi.org/10.1016/j.jss.2025.112480>
13. Zhan, L., Ming, J., Fu, J., Peng, G., Sha, L., Lan, L.: The hidden complexities of android tpl detection: An empirical analysis of techniques, challenges, and effectiveness. *Computers & Security* **159**, 104672 (2025). <https://doi.org/10.1016/j.cose.2025.104672>
14. Zhang, Z., Luo, S., Lu, Y., Pan, L.: Obfuscation-resilient detection of android third-party libraries using multi-scale code dependency fusion. *Information Fusion* **117**, 102908 (2025). <https://doi.org/10.1016/j.inffus.2024.102908>